# The Hidden Bottleneck in MLA Serving: Reconstruction GEMMs, INT4 Quantization, and the L2 Cache Barrier

Robert Zhang[1]

[1]Purdue University
[1]zhan4808@purdue.edu

## Abstract

Multi-head Latent Attention (MLA) compresses KV cache via low-rank latent projections, greatly reducing memory traffic for the attention kernel. We show that this compression creates a *new* dominant bottleneck: the **reconstruction GEMMs**, weight-absorbed batch matrix multiplications that recover full-dimensional K/V from compressed latents, consume **61% of attention-layer time** at batch size 1 on DeepSeek-V3-scale architectures (128 heads, 61 layers), exceeding the attention kernel itself. These BMMs are memory-bound at all practical batch sizes (arithmetic intensity 1–93 vs. H100 roofline crossover at 295).

We investigate INT4 weight-only quantization as a mitigation. Selective INT4 of reconstruction weights preserves quality ($\Delta$PPL = +0.051 on wikitext-2, vs. +6.057 for naive whole-model INT4). However, a custom batched W4A16 Triton kernel achieves only 0.49× of cuBLAS FP16; we identify **two stacked root causes**. First, *partial L2 residency*: warm-state hardware counters (`ncu --cache-control none`) show $\sim$75% of the 16 MB reconstruction weight matrix is served from L2 in steady state, at an effective 3.5–4.6 TB/s, weakening the HBM-bound roofline assumption that motivates quantization. The effective residency capacity is $\sim$36 MB — measured DRAM re-reads collapse at 16–32 MB and snap to 100% at $\geq$40 MB — below the nominal 50 MB L2. Second, *dequantization compute*: a fixed-shape weight-rotation intervention that forces FP16 weights out of L2 shows INT4 *still* loses (1.12× slower), because the W4A16 kernel is SM-bound from dequantization (up to 79% SM utilization) and runs 2.7× above its memory entitlement. L2 residency removes most of the theoretical INT4 upside; dequantization overhead destroys the rest. Scaling weight size from 8 MB to 128 MB, the kernel-level INT4/FP16 gap drops from 1.9× to parity ($\sim$1.1×) — INT4 never wins. We are not aware of prior published characterization of this failure mode.

We validate our profiling methodology through an independent FlashInfer vs. Triton kernel comparison on Llama-3-8B, reproducing known results (89% vs. 80% HBM utilization in decode; 2.6× prefill gap from TMA hardware loads) and contributing new NCU-level root cause analysis. All code and data are publicly available.

## 1 Introduction

Multi-head Latent Attention (MLA) [4] compresses the KV cache through low-rank latent projections, achieving a 7.1× reduction in KV memory traffic and powering frontier models including DeepSeek-V2 and V3 [5]. But this compression requires full-dimensional K and V to be *reconstructed* from compressed latents at inference time via weight-absorbed batch matrix multiplications (BMMs). Prior work has focused on MLA's memory savings; the runtime cost of reconstruction has not been studied.

We show that this cost is the **dominant bottleneck** in MLA attention layers. On DeepSeek-V3-scale architectures (128 heads, 61 layers), reconstruction GEMMs consume 61% of per-layer time at batch size 1, exceeding the attention kernel itself. This overhead is fixed per query, independent of KV length, and invisible to standard attention benchmarks. MLA's compression, in other words, shifts the bottleneck from KV cache reads to weight-matrix reads for reconstruction.

Since reconstruction is memory-bound (arithmetic intensity 1–93 vs. H100 roofline crossover at 295), INT4 weight-only quantization is a natural mitigation. Selective INT4 preserves quality ($\Delta$PPL = +0.051), but a custom Triton W4A16 kernel achieves only 0.49× of cuBLAS FP16, far from the 3.9× predicted by roofline. We trace this to what we term the **L2 cache barrier** *combined with a dequantization-bound kernel*: warm-state counters show $\sim$75% of the 16 MB reconstruction weight matrix is served from L2 (effective 3.5–4.6 TB/s) rather than HBM at 3.35 TB/s, so reducing precision saves HBM bandwidth that was largely not the bottleneck (Figure 6); and even when residency is experimentally removed, the W4A16 kernel's dequantization compute keeps it at or below FP16 parity. Scaling weight size across the L2 boundary, the INT4/FP16 time ratio drops from 1.9× at 8 MB toward parity at 128 MB, with the transition at 32–40 MB — the *effective* residency capacity, below the nominal 50 MB (Figure 8). MLA's compression

paradoxically makes reconstruction weights *too small* to benefit from weight-only quantization.

We validate our profiling methodology through an independent FlashInfer [14] vs. Triton [11] kernel comparison on Llama-3-8B (Section 3), reproducing known performance gaps and contributing new NCU-level root cause analysis.

**Contributions:**

- **Reconstruction GEMM overhead**: first quantification showing 61% of MLA attention-layer time at bs=1 (2.17 ms/token across 61 layers).
- **The L2 cache barrier**: roofline-predicted 3.9× INT4 speedup collapses to 0.49× through two stacked mechanisms — partial L2 residency of the weights (75% L2-served; effective capacity ∼36 MB, effective serving bandwidth 3.5–4.6 TB/s) and a dequantization-bound INT4 kernel that loses to FP16 even when residency is experimentally removed. Validated by a weight-size sweep, warm-state counter analysis (`--cache-control none`), and a fixed-shape weight-rotation intervention. We are not aware of prior published characterization of this failure mode.
- **Selective INT4 quality**: reconstruction weights tolerate INT4 well (+0.051 PPL), establishing them as targets once kernel support matures.
- **NCU-level kernel analysis**: TMA vs. global-load root cause, corrected L1 metrics on Hopper, and the occupancy paradox (fewer warps, more bandwidth).

# 2 Experimental Setup

## 2.1 Hardware

Primary measurements use a single NVIDIA H100 80GB SXM5 (HBM3). Cross-hardware validation uses a single NVIDIA A100 80GB SXM4 (HBM2e). Reference peaks for H100: 3.35 TB/s HBM bandwidth, 989.4 TFLOPS BF16 tensor core, 132 SMs, 50 MB L2 cache. Reference peaks for A100: 2.0 TB/s HBM bandwidth, 312 TFLOPS BF16 tensor core, 108 SMs, 40 MB L2 cache. GPU clocks were not locked on either platform (`nvidia-smi -lgc`); Section 7 quantifies the resulting variance.

## 2.2 Software

SGLang (commit `7ef18be`), PyTorch 2.9.1+cu128, FlashInfer 0.6.6, Triton 3.5.1, CUDA 12.8, NCU 2025.1.1.

## 2.3 Model Configurations

**Model choice rationale.** Llama-3-8B is the standard FlashInfer/Triton benchmark target. DeepSeek-V3 shapes ($H$=128, $d_{\text{lora}}$=512) define the MLA microbenchmarks: the full model is 671B and requires multi-GPU serving, so we instantiate its BMM dimensions directly without loading weights. DS-V2-Lite (15.7B, same MLA architecture) is the smallest single-GPU MLA model available, used for perplexity evaluation. Qwen2.5-7B provides the GQA end-to-end baseline.

Table 1: Configurations used in this work. DS-V3 shapes ($H$=128, $d_{\text{lora}}$=512) are synthetic microbenchmark dimensions only (no weights loaded). DS-V2-Lite is used for perplexity evaluation; Qwen2.5-7B for end-to-end decomposition.

| Model | $H_q$ | $H_{kv}$ | $d$ | Type | KV LoRA |
|---|---|---|---|---|---|
| Llama-3-8B | 32 | 8 | 128 | GQA 4:1 | — |
| DS-V3 shapes | 128 | 128 | 128 | MLA | 512 |
| DS-V2-Lite (15.7B) | 16 | 16 | 128 | MLA | 512 |
| Qwen2.5-7B | 28 | 4 | 128 | GQA 7:1 | — |

## 2.4 Methodology

**Timing.** CUDA events with 10+ warmup iterations, median of 100+ timed iterations. Reproducibility verified across 3 independent trials (Section 7).

**FLOP counting.** Standard attention: $4 \cdot B \cdot \frac{S^2}{2} \cdot H \cdot d$ ($QK^\top$ matmul + AV matmul, $\times 2$ for multiply-accumulate, $\div 2$ for causal masking). Reconstruction uses $M \times K \times N \times 2$ per BMM per head.

**Bandwidth formula.** Decode: $\text{BW} = (\text{KV}_{\text{read}} + Q_{\text{read}} + O_{\text{write}})/t_{\text{median}}$, where KV accounts for 99.8% of bytes.

**NCU profiling.** Collected with `ncu --set full --profile-from-start no` (kernel replay); overhead excluded from bandwidth runs. **Cache-state caveat:** kernel replay defaults to `--cache-control all`, which flushes GPU caches before every measured launch. All replay-based counters in this paper are therefore *cold-cache* measurements; they are valid for kernel classification (DRAM- vs. SM-bound) but carry no information about steady-state cache residency. Residency claims are based exclusively on (a) warm-loop counters collected with `--cache-control none` and (b) timing-only interventions (Section 5.5).

**Timing floor.** Per-launch CUDA-event measurement of these microsecond-scale kernels carries a $\sim$15.5 $\mu$s launch/eventing floor on our stack (a 0.5 MB bmm times identically to a 16 MB one). Where kernel-level conclusions are required we therefore also report CUDA-graph timing (20+ launches captured per graph, median of 50 replays), which removes per-launch CPU overhead.

# 3 Attention Kernel Validation

Before presenting our MLA findings, we validate the profiling methodology by comparing FlashInfer and Triton attention kernels on Llama-3-8B, a well-studied GQA configuration. This reproduces known performance gaps and establishes the measurement infrastructure used in subsequent sections.

Table 2: NCU metrics collected and their interpretation.

| Metric | Interpretation |
|---|---|
| `dram__throughput` | HBM utilization (% peak) |
| `sm__throughput` | SM utilization (% peak) |
| `sm__warps_active` | Active warp occupancy |
| L1/L2 sector hit/miss counts | Cache hierarchy behavior |
| `launch__registers_per_thread` | Register pressure |
| `smsp__inst_executed_pipe_tensor` | Tensor core instruction count |

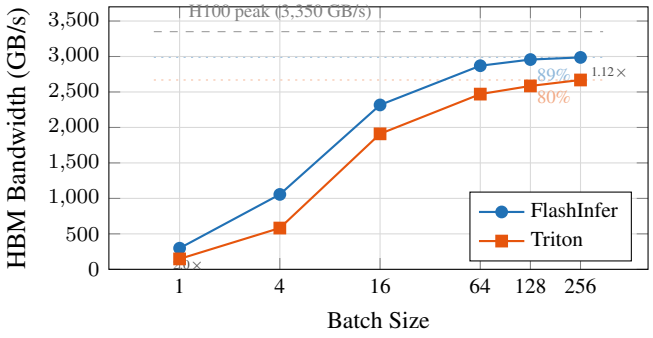


Figure 1: Decode bandwidth vs. batch size (Llama-3-8B, $L_{kv}$=2048). Both backends approach HBM saturation; FlashInfer peaks at 89% of H100 peak. The gap narrows from 2× at bs=1 to 1.12× at bs=256 as launch overhead becomes negligible relative to streaming KV reads.

## 3.1 Decode: Bandwidth Scaling

Table 3: Decode performance, Llama-3-8B GQA, $L_{kv} = 2048$.

| BS | FI (ms) | Tri (ms) | FI BW | Tri BW | Speedup |
|---|---|---|---|---|---|
| 1 | 0.028 | 0.057 | 298 | 147 | **2.03**× |
| 4 | 0.032 | 0.058 | 1056 | 582 | 1.82× |
| 16 | 0.058 | 0.070 | 2317 | 1910 | 1.21× |
| 64 | 0.187 | 0.218 | 2870 | 2470 | 1.16× |
| 128 | 0.364 | 0.416 | 2957 | 2585 | 1.14× |
| 256 | 0.720 | 0.806 | **2987** | **2669** | 1.12× |

Both backends are firmly memory-bound (Table 3). FlashInfer peaks at 2,987 GB/s (89% of 3.35 TB/s) while Triton peaks at 2,669 GB/s (80%). The gap narrows from 2× at bs=1 (launch-overhead dominated) to 1.12× at bs=256 (bandwidth-saturated); Figure 1 shows the full scaling curve.

Table 4: Decode performance, Llama-3-8B GQA, bs=64, varying $L_{kv}$.

| $L_{kv}$ | FI (ms) | Tri (ms) | FI BW | Tri BW | Speedup |
|---|---|---|---|---|---|
| 256 | 0.037 | 0.065 | 1859 | 1053 | **1.76**× |
| 512 | 0.059 | 0.069 | 2273 | 1950 | 1.17× |
| 1024 | 0.102 | 0.113 | 2648 | 2389 | 1.11× |
| 2048 | 0.187 | 0.217 | 2875 | 2476 | 1.16× |
| 4096 | 0.360 | 0.409 | 2984 | 2624 | 1.14× |
| 8192 | 0.716 | 0.790 | **3000** | **2720** | 1.10× |

FlashInfer approaches 3,000 GB/s at $L_{kv} = 8192$, saturating HBM at 89.6% of peak (Table 4).

## 3.2 Decode: NCU Analysis

Table 5: NCU metrics, decode, bs=64, $L_{kv} = 2048$.

| Metric | FlashInfer | Triton (Stage 1) |
|---|---|---|
| Kernel | `BatchPrefill..` | `_fwd_grouped..` |
| DRAM util. | **84%** | 76% |
| SM util. | 17% | 28% |
| Regs/thread | **183** | 76 |
| Active warps | 12.2% | **35.4%** |
| Tensor insts | 2.16M | 2.10M |
| L2 hit rate | 0.4% | 1.0% |
| Duration | **191** $\mu$s | 217 $\mu$s |

**The occupancy paradox.** Table 5 shows FlashInfer uses 183 registers/thread (2.4× Triton's 76), yielding only 12.2% active warps vs. 35.4%. Yet FlashInfer achieves 84% DRAM throughput vs. 76%. This shows that *memory access pattern quality dominates occupancy* for bandwidth-bound kernels: FlashInfer's fused design with coalesced, pipelined accesses extracts more bandwidth per warp than Triton's higher-occupancy two-phase approach.

**Two-phase overhead.** Triton splits attention into Stage 1 (KV scatter-gather, 217 $\mu$s) and Stage 2 (softmax reduction, 10 $\mu$s). Stage 2 adds 4.4% overhead but achieves 73.8% occupancy with only 30 registers, a well-optimized reduction kernel.

**L2 irrelevance at scale.** Both backends show <1% L2 hit rate. The KV cache at bs=64 is $64 \times 2048 \times 8 \times 128 \times 2 = 256$ MB, far exceeding the 50 MB L2. GQA's 4:1 head sharing provides no cache benefit at this scale.

## 3.3 Prefill: TFLOPS Scaling

Table 6: Prefill performance, Llama-3-8B GQA. TFLOPS uses corrected formula ($4BS^2Hd/2$).

| BS | $S$ | FI (ms) | Tri (ms) | FI TF | Speedup |
|---|---|---|---|---|---|
| 1 | 512 | 0.024 | 0.040 | 88 | 1.63× |
| 1 | 1024 | 0.033 | 0.069 | 264 | 2.12× |
| 1 | 2048 | 0.089 | 0.214 | 387 | 2.41× |
| 1 | 4096 | 0.283 | 0.730 | 485 | 2.58× |
| 4 | 2048 | 0.307 | 0.753 | 448 | 2.45× |
| 4 | 4096 | 0.995 | 2.698 | **552** | **2.71**× |
| 16 | 2048 | 1.238 | 2.822 | 444 | 2.28× |
| 16 | 4096 | 4.299 | 10.531 | 512 | 2.45× |

FlashInfer peaks at **552 TFLOPS** (55.8% of 989.4 T peak); Triton peaks at 209 TFLOPS (21.1%) (Table 6, Figure 2). The 2.6× gap is consistent across all configurations and *widens* with sequence length, pointing to a structural difference rather than an incidental one.

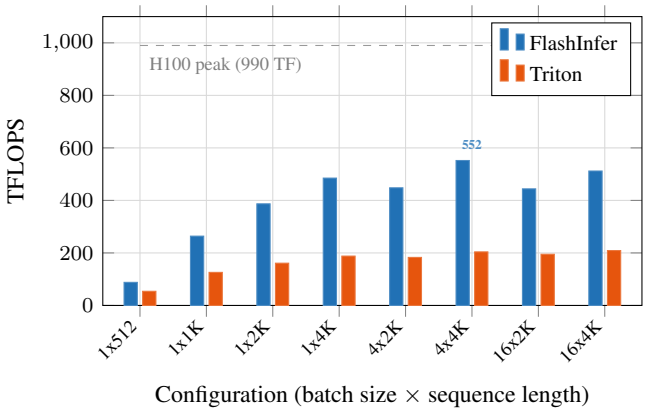


Figure 2: Prefill compute throughput (Llama-3-8B GQA). FlashInfer achieves 2.1–2.7× higher TFLOPS across all configurations, peaking at 552 TFLOPS (56% of peak). The gap widens with sequence length due to FlashInfer's TMA hardware loads and cooperative grid launch (Section 3.4).

Table 7: NCU metrics, prefill, bs=4, $S = 2048$.

| Metric | FlashInfer | Triton |
|---|---|---|
| Kernel | `PrefillWith...` | `_fwd_kernel` |
| SM util. | **52.0%** | 32.7% |
| DRAM util. | 13.1% | 9.3% |
| Regs/thread | 168 | **255** (max) |
| Active warps | 14.1% | 12.5% |
| Grid size | 132 (cooperative) | 2048 |
| L1 global ld sectors | **108K** | 37.8M |
| L2 hit rate | **86.4%** | 72.0% |
| Tensor insts | 2.23M | 2.23M |
| LOCAL mem (CUBIN) | **0 bytes** | **0 bytes** |
| Duration | **368** $\mu$s | 909 $\mu$s |

## 3.4 Prefill: NCU Root Cause Analysis

### 3.4.1 Finding: TMA vs. Global Loads (Not "Cache Hit Rate")

A naive reading of NCU's L1 sector counters (Table 7) yields: FlashInfer 97% L1 hit rate, Triton 0.1%. **This is misleading.**

FlashInfer's Hopper kernel uses TMA (Tensor Memory Accelerator) [10], a dedicated hardware unit that performs asynchronous bulk tensor copies directly from HBM/L2 into shared memory, *bypassing the L1 global load data path entirely*. The 108K L1 global load sectors are residual metadata accesses (indptr arrays, scalar parameters), not QKV tensor data. The 37.8M sectors in Triton represent the *actual* QKV working set flowing through L1 via standard `ld.global` instructions.

The correct comparison is at L2: FlashInfer achieves 86.4% hit rate vs. Triton's 72.0%. FlashInfer's cooperative grid launch (132 blocks = 1 per SM) creates a controlled L2 working set; Triton's 2048-block grid generates 83% more L2 misses (9.9M vs. 5.4M sectors) due to wave-quantization thrashing.

This distinction is architecturally fundamental. TMA access is not "better caching" but a *different hardware data path* unavailable to Triton's compiler, which generates standard `ld.global` PTX.

### 3.4.2 Finding: No Register Spills (Occupancy-Only Bottleneck)

Triton's prefill kernel compiles to 255 registers, the `sm_90` hardware maximum. A natural hypothesis is that registers spill to local memory, adding latency. We tested this at three levels:

1. **NCU local memory counters**: The local memory wavefront counters (`l1tex..mem_local_op_{ld,st}.sum`) returned `n/a` on all Hopper kernels (known NCU limitation on sm_90).
2. **PTX inspection**: Zero `.local` directives, zero `st.local` / `ld.local` instructions in the compiled PTX.
3. **CUBIN resource dump** (`cuobjdump --dump-resource-usage`): `LOCAL:0` for `_fwd_kernel`.

**Verdict**: Triton's 255-register kernel does *not* spill. The compiler fits all live variables within the register file at the cost of maximal register allocation. The performance impact is purely occupancy: fewer warps per SM means fewer opportunities to hide memory latency.

Reducing Triton's register count (e.g., via different tiling or explicit `num_stages` tuning) could improve occupancy and partially close the prefill gap, but the TMA access pattern difference (Section 3.4.1) would remain.

### 3.4.3 Finding: Cooperative Grid vs. Wave Quantization

FlashInfer launches exactly 132 thread blocks (one per SM) using CUDA cooperative launch semantics. This eliminates wave-quantization effects and enables persistent-kernel-style execution where each block processes multiple tiles sequentially with full shared memory and register file utilization.

Triton launches 2,048 blocks (15.5× more), dispatched in multiple waves. Each wave evicts the previous wave's L2 residency, explaining the 14-point L2 hit rate gap (86.4% vs. 72.0%).

# 4 The MLA Reconstruction Bottleneck

Having established confidence in our profiling infrastructure, we turn to our central question: how much of MLA's attention-layer time is spent on reconstruction?

Multi-head Latent Attention [4] compresses KV cache to a low-rank latent $c_t \in \mathbb{R}^{d_{\text{lora}}}$ with $d_{\text{lora}} = 512$. During decode, full-dimensional K and V are *reconstructed* via weight-absorbed batch matrix multiplications (BMMs):

$$\text{BMM1: } Q'_h = Q_h W_h^{\text{kc}}, \quad W_h^{\text{kc}} \in \mathbb{R}^{d_{\text{nope}} \times d_{\text{lora}}}$$
$$\text{BMM2: } O_h = A_h W_h^{\text{vc}}, \quad W_h^{\text{vc}} \in \mathbb{R}^{d_{\text{lora}} \times d_v}$$

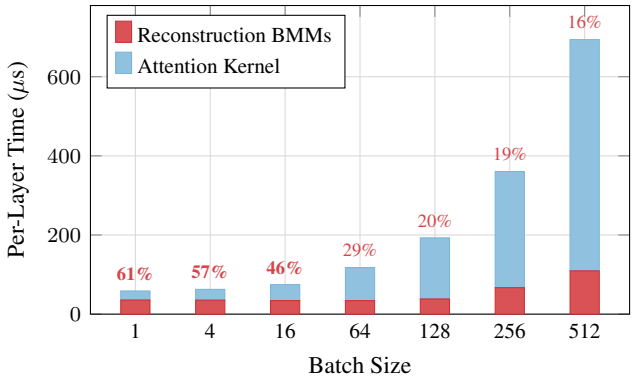


Figure 3: MLA attention-layer time decomposition per layer (DeepSeek-V3 shapes, $L_{kv}$=2048). Reconstruction BMMs (red) dominate at small batch sizes (**61% at bs=1**) and remain significant at bs=512. Reconstruction cost is nearly constant (∼35 $\mu$s), independent of KV length; attention scales linearly.

where $h = 1, \ldots, H$ heads. These are batched GEMMs with batch dimension $H$, independent of KV sequence length.

### 4.1 Cost Breakdown: DeepSeek-V3 Architecture

We profile reconstruction BMMs separately from FlashInfer MLA attention on DeepSeek-V3-scale shapes ($H = 128$, $d_{\text{nope}} = 128$, $d_{\text{lora}} = 512$, $d_v = 128$).

Table 8: MLA reconstruction overhead per layer, DeepSeek-V3 shapes, $L_{kv} = 2048$. Reconstruction cost is independent of KV length.

| BS | Recon ($\mu$s) | Attn ($\mu$s) | Recon % | Model (ms) |
|---|---|---|---|---|
| 1 | **35.6** | 23.0 | **60.7%** | **2.17** |
| 4 | 35.4 | 27.2 | 56.6% | 2.16 |
| 16 | 34.2 | 40.4 | 45.8% | 2.08 |
| 64 | 34.1 | 83.6 | 29.0% | 2.08 |
| 128 | 38.4 | 154.4 | 19.9% | 2.34 |
| 256 | 66.9 | 293.6 | 18.6% | 4.08 |
| 512 | 109.4 | 584.5 | 15.8% | 6.68 |

**Key finding:** At batch size 1, reconstruction BMMs consume **60.7%** of attention-layer time (35.6 $\mu$s vs. 23.0 $\mu$s for the attention kernel itself) (Table 8, Figure 3). Across 61 layers, this totals **2.17 ms per token**, a fixed per-query overhead invisible to standard attention benchmarks.

### 4.2 Roofline Analysis: Memory-Bound at All Batch Sizes

Arithmetic intensity peaks at 93 (bs=128+), well below the crossover point of 295 (Table 9, Figure 4). **All reconstruction BMMs are memory-bound.** Achieved bandwidth ranges from 952 GB/s to 2,114 GB/s (28–63% of HBM peak), leaving significant room for kernel optimization.

Table 9: Arithmetic intensity and achieved bandwidth for reconstruction BMMs. H100 roofline crossover: AI = 295 (990 TFLOPS / 3.35 TB/s).

| BS | AI (BMM1) | $BW_1$ | AI (BMM2) | $BW_2$ |
|---|---|---|---|---|
| 1 | **0.97** | 954 | **0.97** | 952 |
| 16 | 15.5 | 1133 | 15.5 | 1137 |
| 64 | 62.1 | 1592 | 62.1 | 1607 |
| 128 | **93.0** | **1837** | **93.0** | **2114** |
| 256 | 93.0 | 1741 | 93.0 | 1770 |

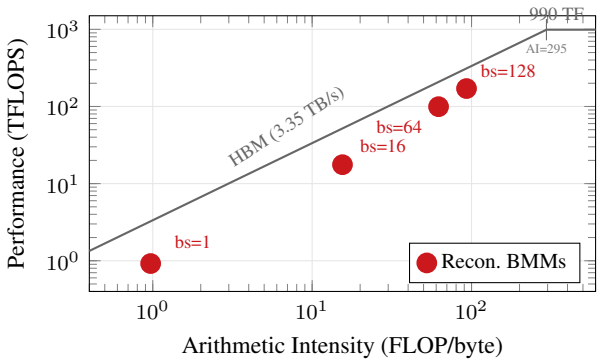


Figure 4: H100 roofline with MLA reconstruction BMMs. All operating points fall on the memory-bound slope, well below the compute ceiling (crossover at AI=295). Even at bs=128 (AI=93), reconstruction achieves only 55% of the memory ceiling.

**Critical implication:** Because reconstruction is memory-bound and weight-dominated (weight matrix is $128 \times 128 \times 512 \times 2 = 16$ MB per BMM, fixed regardless of batch size), reducing weight precision directly reduces the memory bandwidth bottleneck.

## 5 INT4 Mitigation and the L2 Cache Barrier

Reconstruction BMMs are memory-bound at all practical batch sizes, and their weight matrices dominate the data transfer. Reducing weight precision should therefore yield proportional speedups. We evaluate INT4 weight-only quantization of $W^{\text{kc}}$ and $W^{\text{vc}}$, addressing two questions: does it preserve quality, and does it deliver the expected performance gain?

### 5.1 Perplexity Evaluation

We evaluate on DeepSeek-V2-Lite (15.7B parameters) using wikitext-2 test set (308K tokens, stride-512 sliding window, max length 2048). Three configurations:

**Selective INT4 quantization of reconstruction weights adds only +0.051 perplexity** (Table 10, Figure 5), an order of magnitude below the typical 0.5 quality threshold. Naive INT4 of all linear weights, by contrast, more than doubles perplexity.

Table 10: Wikitext-2 perplexity under INT4 quantization. `kv_b_proj` is the reconstruction weight ($W^{kc}$/$W^{vc}$ combined).

| Configuration | PPL | Δ | Weights quantized |
|---|---|---|---|
| FP16 baseline | **5.727** | — | 0 |
| INT4 selective | 5.777 | **+0.051** | 27 (kv_b_proj) |
| INT4 all linear | **11.784** | +6.057 | 5,209 |

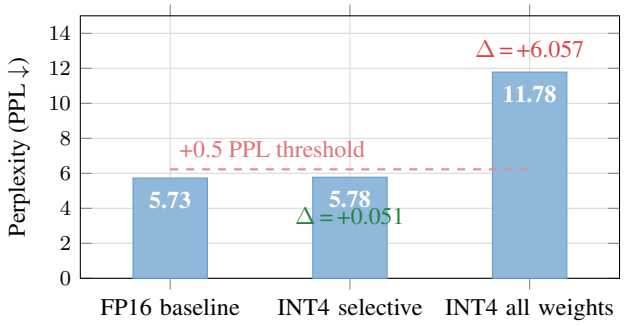


Figure 5: Wikitext-2 perplexity under INT4 quantization (DeepSeek-V2-Lite). Selective INT4 of reconstruction weights (`kv_b_proj`) adds only +0.051 PPL, well within the +0.5 quality threshold. Naive INT4 of all linear weights severely degrades quality.

## 5.2 Why Reconstruction Weights Are Quantization-Friendly

The $W^{kc}$ and $W^{vc}$ matrices are *projection weights* that map between a compressed 512-dimensional latent space and 128-dimensional head spaces. Several factors explain their quantization robustness:

1. **Low-rank structure**: The latent space was trained to be compressible; the projection matrices inherit smooth spectral properties.
2. **Redundancy**: With $H = 128$ heads each projecting from the same 512-dim latent, per-head reconstruction errors are averaged across heads before affecting output.
3. **Post-softmax attenuation (hypothesized)**: BMM2 operates on attention-weighted values; quantization noise is attenuated by the softmax distribution's sparsity.

## 5.3 Theoretical Speedup

For memory-bound operations, speedup from weight quantization equals the ratio of total data transferred:

$$\text{Speedup} = \frac{\text{Weight}_{\text{FP16}} + \text{Activation}_{\text{FP16}}}{\text{Weight}_{\text{INT4}} + \text{Scale/ZP} + \text{Activation}_{\text{FP16}}} \quad (1)$$

At batch size 1 (latency-critical single-query serving), the roofline predicts **3.94× speedup** (Table 11), which would reduce the 2.17 ms full-model reconstruction overhead to ∼0.55 ms.

Table 11: Theoretical INT4 speedup for reconstruction BMMs. Weight bytes dominate at small batch sizes.

| BS | Wt (MB) | Act (MB) | INT4 eq. (MB) | Speedup |
|---|---|---|---|---|
| 1 | 16.00 | 0.03 | 4.00 | **3.94×** |
| 4 | 16.00 | 0.13 | 4.00 | 3.89× |
| 16 | 16.00 | 0.50 | 4.00 | 3.67× |
| 64 | 16.00 | 2.00 | 4.00 | 3.00× |
| 128 | 16.00 | 4.00 | 4.00 | 2.50× |
| 256 | 16.00 | 8.00 | 4.00 | 2.00× |

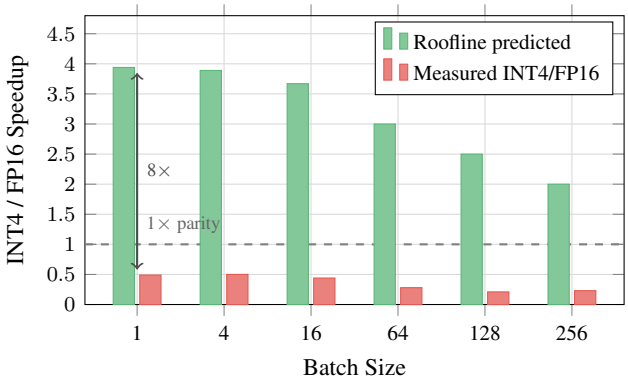


Figure 6: The L2 cache barrier. Roofline analysis predicts 2–3.9× INT4 speedup assuming HBM-bound operation. Measured performance shows 0.2–0.5×: the INT4 kernel is *slower* than cuBLAS FP16. The 16 MB reconstruction weight fits in H100's 50 MB L2; after first access, weights are served from L2 at ∼12 TB/s, making HBM savings irrelevant.

## 5.4 Measured Kernel Performance

We implemented a custom batched W4A16 Triton kernel that fuses the head dimension into the grid, avoiding 128 separate kernel launches. The kernel dequantizes INT4 weights to FP16 in registers and uses tensor core `tl.dot` for the matmul.

Table 12: Measured INT4 kernel performance vs. cuBLAS FP16 `torch.bmm`. "FP16 loop" = 128 individual `torch.mm` calls.

| BS | FP16 bmm (ms) | INT4 Tri (ms) | FP16 loop (ms) | Ratio INT4/FP16 |
|---|---|---|---|---|
| 1 | **0.036** | 0.073 | 2.194 | **0.49×** |
| 4 | 0.037 | 0.073 | 2.203 | 0.50× |
| 16 | 0.036 | 0.082 | 2.187 | 0.44× |
| 64 | 0.036 | 0.129 | 2.215 | 0.28× |
| 128 | 0.040 | 0.187 | 2.211 | **0.21×** |
| 256 | 0.070 | 0.302 | 2.223 | 0.23× |

**The INT4 kernel is 2× slower than cuBLAS FP16, not 3.9× faster** (Table 12, Figure 6). It is, however, 30× faster than a per-head FP16 loop (0.073 ms vs. 2.19 ms), confirming that the batched grid approach itself is effective. The bottleneck is competing with cuBLAS, not the batching strategy.

## 5.5 The L2 Cache Barrier: Why Roofline Fails for Small-Matrix MLA Reconstruction

The 8× gap between roofline-predicted (3.9×) and measured (0.49×) performance has three contributing factors. No single factor is sufficient: a fixed-shape rotation intervention (below) shows that removing factor (1) alone still leaves INT4 at or below FP16 parity.

**(1) Partial L2 residency weakens the HBM-bound assumption.** The standard roofline model [13] assumes data is served from HBM at 3.35 TB/s. But the total reconstruction weight per BMM is $128 \times 128 \times 512 \times 2 = 16$ MB, within H100's 50 MB L2 cache. Warm-loop counters (`ncu --cache-control none`, steady-state launch after 30 warm-ups) show `torch.bmm` re-reads only 4.0 MB of the 16.8 MB working set from DRAM per launch — ∼75% L2-served — at an effective serving bandwidth of 3.5–4.6 TB/s. (The often-quoted ∼12 TB/s aggregate L2 bandwidth is not achieved by these kernels; CUDA-graph timing puts the incremental L2-era slope at 6.3 TB/s and total-bytes throughput at 3.5–4.6 TB/s.) Reducing weight precision from FP16 to INT4 saves HBM bandwidth that *was largely not the bottleneck*.

We validate the size dependence by scaling the weight matrix across the L2 boundary. Holding $H$=128 and $d_{\text{nope}}$=128 fixed, we sweep $d_{\text{lora}}$ from 256 to 4096, producing FP16 weight sizes from 8 MB to 128 MB. Figure 8 shows the event-timed result: the INT4/FP16 time ratio drops from 1.91× at 8 MB to 1.08× at 128 MB. Warm-state DRAM-read counters locate the actual residency cliff at **32–40 MB** (re-reads collapse to 2.2 MB at 32 MB weights, then snap to ∼100% of weight size at 40 MB and above): the *effective* LRU residency capacity is ∼36 MB, below the nominal 50 MB, and the knee in Figure 8 should be read against that corrected boundary. Two confounds inflate the event-timed curve's flatness below 32 MB: a ∼15.5 $\mu$s per-launch timing floor (Section 2.4), and cuBLAS switching `nvjet` tile variants across the sweep. CUDA-graph timing removes the floor and shows FP16 latency scaling smoothly at 6.3 TB/s below the cliff and 2.7 TB/s above it, with kernel-level INT4/FP16 ratios of 1.9× (16 MB) → 1.0–1.1× (≥40 MB): **INT4 reaches parity above the cliff but never wins**.

**Weight-rotation intervention (residency removed at fixed shape).** To test whether L2 residency is *sufficient* to explain INT4's failure, we hold the MLA shape fixed (16 MB weights) and cycle each launch through $k$ independent weight copies inside a CUDA graph. With $k$=1–2 (working set ≤32 MB, L2-resident) FP16 takes 4.8 $\mu$s per BMM; with $k$=6 (96 MB working set, residency destroyed) FP16 rises to 8.3 $\mu$s — the +3.5 $\mu$s delta matching an HBM-rate weight reload. INT4 is cache-state-insensitive (9.1 vs. 9.3 $\mu$s). Critically, **INT4 remains 1.12× slower than FP16 even with FP16 forced to HBM**. L2 residency accounts for the gap growing from 1.12× to 1.91×, but eliminating it would not make this INT4 kernel win: the kernel runs 2.7× above its memory entitlement (∼3.4 $\mu$s for 4.2 MB at HBM rate) due to factor (2).

**Controlled cache-residency intervention.** To establish L2 residency as a causal factor rather than a correlate of matrix size, we hold the reconstruction shape fixed ($H$=128, $d_{\text{nope}}$=128, $d_{\text{lora}}$=512) and vary only the cache state immediately before each timed launch. Under the warm condition, 20 untimed kernel executions load the 16 MB weight tensor into L2 before measurement. Under the cold condition, a memory sweep (4× L2 capacity: 256 MB on H100, 192 MB on A100) evicts L2 contents immediately before the timed launch, with explicit synchronization between eviction and measurement.

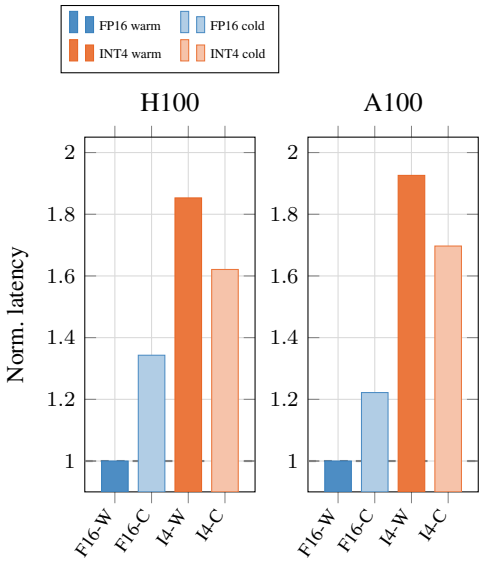


Figure 7: Cache-residency intervention at fixed MLA reconstruction shape (BS=1, normalized to FP16-warm). H100: eviction adds +4.8 $\mu$s to FP16 (= full 16 MB weight reload at HBM rate) and +3.9 $\mu$s to INT4, of which ∼2.6 $\mu$s is common-mode eviction overhead (expected INT4 reload: 1.3 $\mu$s). The FP16 delta supports L2-residency dependence of the FP16 path; note that absolute deltas, not relative percentages, are the comparable quantity here since the two baselines differ ∼2.6×.

Figure 7 shows this intervention on both H100 and A100. We report *absolute* latency deltas, since the FP16 and INT4 baselines differ by ∼2.6× and relative percentages are therefore not comparable across kernels. On H100, forced eviction adds +4.8 $\mu$s to FP16 — matching a full 16 MB weight reload at HBM rate (16 MB / 3.35 TB/s = 4.8 $\mu$s) — and +3.9 $\mu$s to INT4. INT4's expected reload is only ∼1.3 $\mu$s (4.2 MB packed), so roughly +2.6 $\mu$s of its delta is common-mode overhead from the eviction procedure itself (DRAM write-back of the dirty eviction buffer); a control with an 8 MB eviction buffer (too small to displace the weights) adds <0.3 $\mu$s to FP16, confirming the eviction buffer, not measurement noise, produces the common-mode term. The FP16-specific component (+4.8 $\mu$s ≈ exact HBM reload time) supports L2-residency dependence of the FP16 path; an earlier version of this analysis reported the same data as relative percentages (FP16 +34% vs. INT4 +17%), which overstated the asymmetry. NCU counter attribution per tensor allocation is not available in current tooling; the warm-loop `--cache-control none` DRAM-read counters and the rotation intervention above are the primary causal evidence,

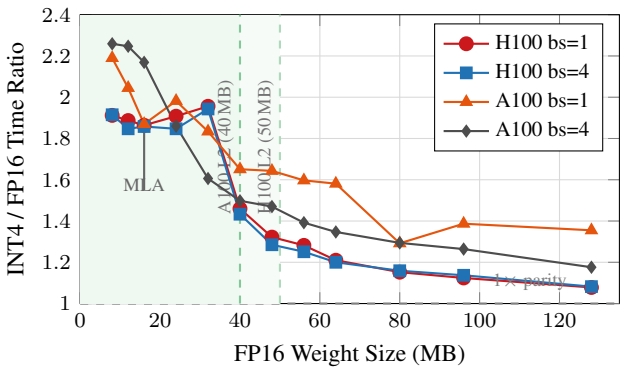


Figure 8: Weight-size scaling on H100 and A100: event-timed INT4/FP16 ratio vs. FP16 weight size. Below the knee INT4 is much slower; above it the ratio moves toward (but never below) parity as FP16 falls back to HBM. Two caveats apply: (i) the H100 knee sits at 32–40 MB, the *effective* residency capacity measured by warm-state DRAM-read counters, not at the nominal 50 MB; (ii) these event-timed curves embed a $\sim$15.5 $\mu$s per-launch floor that flattens the small-size region — CUDA-graph timing shows the same knee with kernel-level ratios of 1.9× $\rightarrow$ 1.0–1.1×. At MLA's 16 MB operating point (arrow), both platforms are in the (partially) L2-resident regime.

with this eviction experiment as corroboration.

Replay-based NCU profiling across the same sweep classifies the two kernels — with the caveat that these counters are *cold-cache* (cache-flushed replay, Section 2.4) and therefore speak to kernel character, not to residency. On H100, the FP16 cuBLAS kernel (`nvjet`, TMA-based) is memory-side at every size: SM utilization stays below 15%, and DRAM utilization rises smoothly from 35% at 8 MB to 83% at 128 MB — a duration-amortization effect (short kernels cannot sustain peak DRAM), *not* an L2-boundary knee; the steepest rise occurs below 32 MB. The INT4 Triton kernel is the opposite at every size: SM utilization scales from 33% to 79% (dequantization dominates) while DRAM utilization stays below 23%. On A100 the directions are the same (FP16 DRAM 43%$\rightarrow$66%, INT4 SM 51%$\rightarrow$75%). There is thus no within-kernel memory-to-compute *transition* anywhere in the sweep: FP16 is always memory-side and INT4 always dequant-compute-side; what changes with size is FP16's serving *tier* (L2 below $\sim$36 MB effective capacity, HBM above), which is visible only in the warm-state counters and timing interventions above. INT4 reads exactly 4× fewer DRAM bytes as expected, but converting that savings into latency requires the kernel to not be compute-bound, which it is.

This creates a paradox specific to MLA: the same low-rank compression that makes reconstruction weights small enough to be a latency concern *also* makes them small enough to be L2-resident, defeating the primary motivation for weight quantization. Standard LLM linear layers (e.g., FFN projections with weights in the hundreds of MB) do not exhibit this behavior.

**(2) INT4 dequantization shifts the bottleneck from memory to compute.** The NCU data shows that the INT4 kernel is SM-bound (up to 79% SM utilization) rather than DRAM-bound. Bit masking, shifting, signed extension, and type conversion on every packed byte consume the freed bandwidth headroom. Even when weights exceed L2 and FP16 falls back to HBM, INT4 cannot fully exploit its 4× data reduction because dequantization saturates the SMs first.

**(3) cuBLAS dispatch advantage.** `torch.bmm` dispatches via the `nvjet` kernel family with TMA hardware loads (168 registers, L1 bypassed entirely), while our Triton kernel uses standard `ld.global` with 57 registers. Under concurrent L2 contention, INT4 shows greater sensitivity to cache pressure than FP16—consistent with its irregular packed-byte access pattern.

**When does INT4 help?** The L2 cache barrier is not absolute. In production serving, concurrent operations (FFN GEMMs, attention across multiple layers, multi-tenant request batching) contend for L2 capacity and may evict reconstruction weights back to HBM. However, our rotation intervention bounds what this can buy for the *current* kernel: with FP16 fully evicted, this W4A16 implementation only reaches parity (1.12× slower at the 16 MB point; 1.0–1.1× across larger sizes). Realizing an actual win under L2 pressure additionally requires a dequantization path that does not saturate the SMs — e.g., INT8 tensor-core MMA with fused dequant — after which fusing reconstruction across layers to exceed L2 capacity, or deploying on GPUs with smaller L2, would let the 4× byte reduction translate into latency.

## 5.6 A Measured Cache-Aware Roofline

The failures above are failures of the *single-ceiling* roofline. We therefore construct a cache-aware roofline in the spirit of Ilic et al. [8], with two extensions required for microsecond-scale LLM kernels, and with every parameter *measured* on the target GPU (`measure_carm_params.py`) rather than taken from datasheets:

$$t = t_0 + \max\left(\frac{B}{\mathrm{BW}(\mathrm{WS})}, \frac{F}{P_{\mathrm{peak}}}\right), \qquad \mathrm{BW}(\mathrm{WS}) = \begin{cases} \mathrm{BW}_{L2}^{\mathrm{eff}} & \mathrm{WS} < \\ \mathrm{BW}_{\mathrm{HBM}}^{\mathrm{eff}} & \text{otherwi} \end{cases} \tag{2}$$

**Extension 1: capacity-gated bandwidth with effective parameters.** The bandwidth ceiling is a function of working-set size, gated at the *measured* effective residency capacity $C_{\mathrm{eff}} \approx 36$ MB (not the nominal 50 MB), with measured tier bandwidths $\mathrm{BW}_{L2}^{\mathrm{eff}} = 5.0$–$6.3$ TB/s (reduction- and GEMM-pattern, respectively; not the $\sim$12 TB/s aggregate figure) and $\mathrm{BW}_{\mathrm{HBM}}^{\mathrm{eff}} = 3.15$ TB/s.

**Extension 2: an explicit per-launch fixed cost** $t_0$ (measured: 2.8 $\mu$s graph-captured, 15.4 $\mu$s eager). Classic rooflines plot performance against arithmetic intensity alone; for kernels whose data-path time is a few microseconds, $t_0$ dominates and every small-batch operating point sits far below the loft for reasons no bandwidth ceiling can express. In Figure 9 (panel A) we draw a per-point launch-floor cap $F/t_0$ that makes this regime explicit.

The INT4 kernel requires a different ceiling: it is bounded by dequantization throughput, not bandwidth. Fitting $t = t'_0 + W_{\text{packed}}\lceil \text{bs}/16 \rceil / R_{\text{dq}}$ to the measured points gives $R_{\text{dq}} \approx 0.50$ TB/s of packed bytes, equivalent to an *in-core ceiling of* $\sim$30 *TFLOPS — 3% of peak —* which the INT4 operating points pin against at every batch size (Figure 9, panel A). This single number explains the entire INT4 failure: no amount of bandwidth savings can help a kernel whose effective compute ceiling is 3% of the machine.

The combined model predicts FP16 latency across batch sizes 1–512 with 10% MAPE and INT4 with 18% (Figure 9, panel C); the largest INT4 errors are at bs$\leq$16 where the single-M-tile dequant overlaps the fixed cost, making the model conservative. Both the standard roofline's 3.9$\times$ INT4 prediction and the original "L2 at 12 TB/s" explanation are special cases of this model with wrong parameters: the former ignores $\text{BW}(\text{WS})$ and $R_{\text{dq}}$, the latter overestimates both tiers.

# 6 End-to-End Context

To quantify the system-level impact of reconstruction, we contrast MLA against a standard GQA decode workload.

Table 13: Decode step decomposition, Qwen2.5-7B, bs=64.

| Component | Time ($\mu$s) | Share |
|---|---|---|
| **Linear layers (GEMMs)** | **5,079** | **70.6%** |
| Attention (FlashInfer) | 1,413 | 19.6% |
| Norms (RMSNorm) | 145 | 2.0% |
| Activation (SiLU) | 98 | 1.4% |
| RoPE | 57 | 0.8% |
| Other | 399 | 5.5% |

Attention constitutes $\sim$20% of decode time for GQA models (Table 13, Figure 10). For MLA models like DeepSeek-V3, reconstruction overhead adds a further component: at bs=1, reconstruction GEMMs (2.17 ms across 61 layers) exceed the attention kernel cost (1.40 ms), making reconstruction the dominant attention-layer bottleneck. Targeted optimization of reconstruction, whether through INT4 quantization or kernel fusion, is therefore higher-impact than further attention kernel tuning.

# 7 Reproducibility

Table 14: Reproducibility across 3 independent trials (warmup=20, iters=200).

| Config | T1 | T2 | T3 | CV |
|---|---|---|---|---|
| FI dec bs=1 | 0.029 | 0.030 | 0.030 | 3.4% |
| Tri dec bs=1 | 0.056 | 0.059 | 0.057 | 2.6% |
| FI dec bs=64 | 0.187 | 0.187 | 0.187 | **<0.1%** |
| Tri dec bs=64 | 0.216 | 0.216 | 0.216 | **<0.1%** |
| FI dec bs=256 | 0.719 | 0.718 | 0.718 | **<0.1%** |
| Tri dec bs=256 | 0.813 | 0.813 | 0.813 | **<0.1%** |

At bs$\geq$64, results are deterministic ($<$0.1% CV) (Table 14). At bs=1, $\sim$3% variance arises from GPU boost clock fluctuations on short ($\sim$28 $\mu$s) kernels. Relative speedup ratios are stable across all trials.

# 8 Discussion

Our analysis reveals a chain of findings: MLA's KV compression shifts the attention-layer bottleneck to reconstruction GEMMs (61% at bs=1); these are memory-bound at all practical batch sizes; INT4 quantization preserves quality but fails to deliver speedup due to L2 cache residency. We now discuss the implications.

## 8.1 Implications for Compiler-Generated Attention

The prefill gap is fundamental to the compiler architecture: Triton's MLIR$\rightarrow$PTX pipeline generates `ld.global` instructions because it has no TMA code generation path for attention patterns. Triton 3.x introduced `tl.experimental_descriptor_load` for TMA access, but this requires manual descriptor setup that attention kernels have not adopted. Closing the prefill gap requires either:

1. Triton compiler support for automatic TMA lowering of attention-shaped loads (research problem).
2. Manual TMA integration in the Triton kernel source (engineering, loses portability).
3. A higher-level DSL that targets both TMA and global load paths depending on the GPU architecture.

The decode gap (1.1$\times$) is more tractable: Triton's two-phase split could be fused, and register tuning (`num_warps`, tiling factors) could recover some bandwidth.

## 8.2 The Occupancy Paradox as Design Principle

FlashInfer's decode kernel shows that for memory-bound workloads, *maximizing bandwidth per warp* outperforms *maximizing warp count*. At 183 registers, FlashInfer has 2.4$\times$ fewer active warps than Triton, yet extracts 10% more HBM bandwidth. This is consistent with the principle that memory-bound kernels benefit more from coalesced access patterns and prefetching than from raw occupancy [12].

## 8.3 MLA: Compression Shifts the Bottleneck

MLA's 7.1$\times$ KV traffic reduction makes the attention kernel faster but creates a new bottleneck in the reconstruction GEMMs that were previously negligible. At bs=1, reconstruction costs 61% of attention-layer time. This illustrates a general systems principle: optimizations that reduce data movement

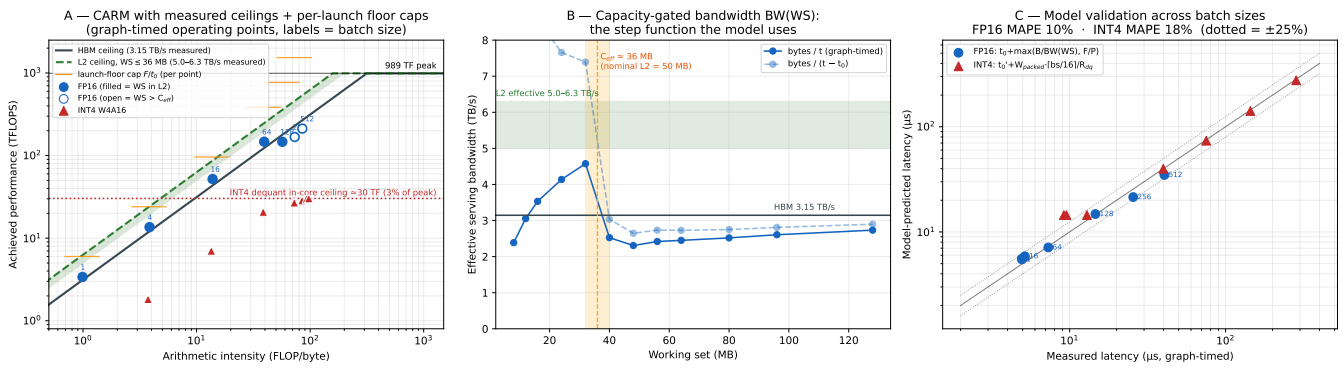


Figure 9: Measured cache-aware roofline for MLA reconstruction (H100, graph-timed). **A**: CARM with measured HBM and L2 ceilings, per-point launch-floor caps, and the fitted INT4 dequant in-core ceiling (∼30 TF); FP16 points (labeled by batch size) are filled when the working set fits $C_{\text{eff}}$. **B**: the capacity-gated bandwidth function $\mathrm{BW}(\mathrm{WS})$ — effective serving bandwidth vs. working set, with the measured residency cliff at 32–40 MB. **C**: model validation, predicted vs. measured latency (dotted $= \pm 25\%$).

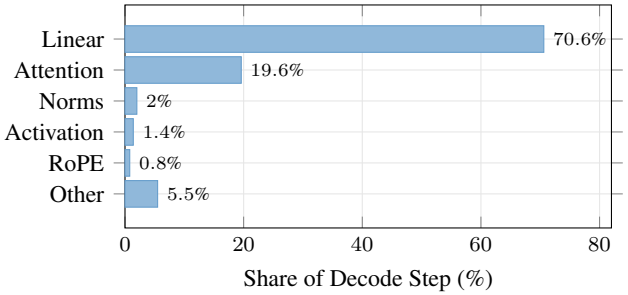


Figure 10: Decode step decomposition (Qwen2.5-7B, bs=64). Linear layers dominate at 71%; attention is ∼20%. For MLA, reconstruction BMMs add to this: at bs=1, reconstruction (2.17 ms) exceeds attention (1.40 ms).

can shift workloads into regimes where cache hierarchy, rather than raw bandwidth, determines performance. The implication for MLA specifically is that reconstruction overhead must be co-optimized with attention, not ignored.

## 8.4 Selective Quantization: Quality Result

Despite the kernel limitations, reconstruction weights tolerate quantization well: +0.051 PPL for selective INT4 vs. +6.057 for naive INT4 of all weights. For future MLA serving systems, **reconstruction weights are the natural first target for aggressive quantization**, pending kernel support.

## 8.5 Limitations

- Single GPU; multi-GPU tensor-parallel workloads may show different ratios due to all-reduce overlap.
- GPU clocks not locked; absolute numbers carry ∼3% uncertainty at small batch sizes.
- Per-launch CUDA-event timing carries a ∼15.5 $\mu$s floor; eager-mode bs≤16 latencies (and the 61% reconstruction share) are launch-overhead-inflated. Serving engines that capture decode in CUDA graphs will see smaller absolute reconstruction overhead (graph-timed: ∼4.8 $\mu$s per BMM vs. ∼17 $\mu$s eager), though the recon-vs-attention ratio is less affected since both sides carry the floor.
- Replay-based NCU counters are cold-cache (`--cache-control all`); only the `--cache-control none` warm-loop counters and timing interventions speak to residency.
- INT4/FP16 ratios are software-stack-sensitive: PyTorch 2.9.1/Triton 3.5.1 yields 1.86× (16 MB) → 1.08× (128 MB) event-timed, while PyTorch 2.7/Triton 3.3 yields 2.75× → 1.58× on identical hardware. Kernel-level (graph-timed) ratios are more stable (1.9× → 1.0–1.1×). Directional conclusions transfer; magnitudes do not.
- cuBLAS dispatches different `nvjet` tile variants across the weight-size sweep; the FP16 curve mixes kernel implementations.
- NCU local memory counters unavailable on Hopper (`n/a`); spill analysis relies on PTX/CUBIN inspection.
- INT4 kernel benchmarked in Triton only; a CUDA-native implementation with INT8 tensor cores may achieve different results.
- Perplexity evaluated on DeepSeek-V2-Lite (15.7B); larger models may show different quantization sensitivity.
- One model architecture per attention type.

# 9 Related Work

**Attention kernels.** FlashAttention [3] and FlashAttention-2 [2] introduced tiling-based exact attention with $O(N)$ memory. FlashInfer [14] extends this with Hopper-native TMA support. Triton [11] enables Python-level GPU kernel development. SGLang [15] integrates both backends for LLM serving.

**MLA architectures.** DeepSeek-V2 [4] introduced Multi-head Latent Attention; DeepSeek-V3 [5] scaled it to 671B parameters with 128 attention heads. While both papers describe the weight-absorbed reconstruction mechanism, neither quantifies its runtime cost relative to the attention kernel.

**Performance modeling.** The roofline model [13] is the standard framework for reasoning about compute- vs. memory-bound kernels. Ilic et al. [8] extended it to cache-aware form with per-level bandwidth ceilings. Our L2 cache barrier finding (Section 5.5) exposes a failure mode of single-level roofline analysis — when working sets (partially) fit in L2, the effective bandwidth ceiling shifts from HBM toward L2 — and Section 5.6 contributes a measured CARM variant with a capacity-gated ceiling, an explicit per-launch fixed cost, and a fitted dequantization in-core ceiling, validated to 10–18% MAPE on the reconstruction kernels. Volkov [12] showed that occupancy is not always the primary performance factor, motivating our analysis of FlashInfer's register-heavy approach.

**Weight quantization.** GPTQ [6] and AWQ [9] enable post-training INT4 weight quantization for LLM inference. KVQuant [7] applies quantization to the KV cache itself, targeting the memory footprint that MLA addresses architecturally. QuaRot [1] uses rotation to eliminate outliers before 4-bit quantization. All prior quantization work targets standard linear layers (FFN, attention projections), not the reconstruction matrices specific to MLA. To our knowledge, no prior public work quantifies the reconstruction GEMM overhead in MLA or evaluates selective INT4 quantization of these weights.

# 10 Conclusion

MLA's reconstruction GEMMs consume 61% of eager-mode attention-layer time at batch size 1—a bottleneck that standard benchmarks miss, that single-level roofline analysis mispredicts by 8×, and that naive INT4 quantization worsens rather than fixes. We also provide a kernel-level characterization of FlashInfer vs. Triton attention on H100, identifying TMA hardware loads, memory access pattern quality, and cooperative grid launch as the three root causes of the performance gap.

The INT4 quantization results illustrate a subtlety in MLA optimization. **Quality is safe**: selective INT4 adds only +0.051 perplexity. **But speedup is not automatic**: our custom Triton W4A16 kernel achieves 0.49× of cuBLAS FP16 through two stacked mechanisms — ∼75% of the 16 MB reconstruction weights are L2-served in steady state (effective capacity ∼36 MB, effective bandwidth 3.5–4.6 TB/s), removing most of the HBM savings INT4 targets; and the dequantization-bound kernel forfeits the remainder, losing to FP16 even when residency is experimentally removed. The gap between roofline-predicted (3.9×) and realized performance is specific to small, (partially) L2-resident weight matrices, a regime common in MLA but absent from standard LLM linear layers. Closing this gap requires CUDA-native INT8 tensor core kernels (fixing the dequant path) *and* either cross-layer weight fusion beyond L2 capacity or deployment under L2 pressure — fixing residency alone is provably insufficient for this kernel.

All profiling scripts, raw data, and the perplexity evaluation code are publicly available for reproduction.

# References


[1] Saleh Ashkboos, Amirkeivan Mohtashami, Maximilian L. Croci, Bo Li, Martin Jaggi, Dan Alistarh, Torsten Hoefler, and James Hensman. Quarot: Outlier-free 4-bit inference in rotated llms. *arXiv preprint arXiv:2404.00456*, 2024.

[2] Tri Dao. Flashattention-2: Faster attention with better parallelism and work partitioning. In *ICLR*, 2024.

[3] Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. In *NeurIPS*, 2022.

[4] DeepSeek-AI. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. *arXiv preprint arXiv:2405.04434*, 2024.

[5] DeepSeek-AI. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.

[6] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. Gptq: Accurate post-training quantization for generative pre-trained transformers. In *ICLR*, 2023.

[7] Coleman Hooper, Sehoon Kim, Hiva Mohammadi, Michael W. Mahoney, Yakun Sophia Shao, Kurt Keutzer, and Amir Gholami. Kvquant: Towards 10 million context length llm inference with kv cache quantization. *arXiv preprint arXiv:2401.18079*, 2024.

[8] Aleksandar Ilic, Frederico Pratas, and Leonel Sousa. Cache-aware roofline model: Upgrading the loft. *IEEE Computer Architecture Letters*, 13(1):21–24, 2014.

[9] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. Awq: Activation-aware weight quantization for llm compression and acceleration. In *MLSys*, 2024.

[10] NVIDIA. Nvidia h100 tensor core gpu architecture. `https://resources.nvidia.com/en-us-tensor-core`, 2022.

[11] Philippe Tillet, H. T. Kung, and David Cox. Triton: An intermediate language and compiler for tiled neural network computations. In *MAPL*, 2019.

[12] Vasily Volkov. Better performance at lower occupancy. In *GPU Technology Conference*, 2010.

[13] Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009.

[14] Zihao Ye, Lequn Lai, Ruihang Shao, Wuwei Zheng, and Tianqi Chen. Flashinfer: Efficient and customizable attention engine for llm inference serving. In *MLSys*, 2025.

[15] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Jeff Huang, Chuyue Sun, Cody Hao Yu, Shiyi Cao, Christos Kober, Yineng Shi, et al. Sglang: Efficient execution of structured language model programs. In *NeurIPS*, 2024.